

Replication

❑ **Suppose your MongoDB server crashes. What happens?**

Users cannot access data.

Application may go down.

New data cannot be written.

❑ **What if we keep copies of the same database on multiple servers?**

This idea is called Replication.

Definition:

Replication is the process of maintaining multiple copies of the same data across multiple MongoDB servers to ensure high availability and fault tolerance.

Client Application

Driver

Write

Primary

Replication

Secondary

Replication

Secondary

Read with Read Preference
secondary

```
graph TD; CA[Client Application] --- D[Driver]; D -- Write --> P[Primary]; P -.->|Replication| S1[Secondary]; P -.->|Replication| S2[Secondary]; R[Read with Read Preference secondary] --> S2;
```


Why Do We Need Replication?

High Availability

If one server fails, another server can take over.

Fault Tolerance

Hardware failures don't bring down the application.

Disaster Recovery

Data can still be recovered from replicas.

Read Scaling

Some read operations can be served by secondary nodes.

What is a Replica Set?

MongoDB replication is implemented using a Replica Set.

Definition:

A Replica Set is a group of MongoDB servers that maintain the same dataset.

Example:

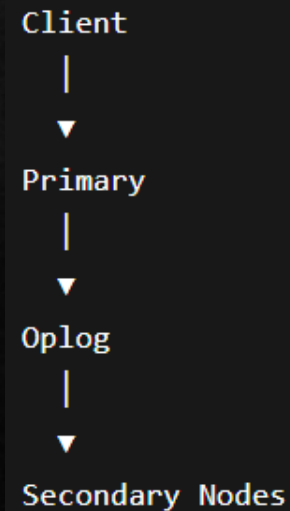
```
Replica Set
|
├─ Primary
├─ Secondary
└─ Secondary
```

How Replication Works Internally

MongoDB uses a special collection called **oplog (Operation Log)**

Every write operation is recorded here. Example – Insert User, Update User, Delete User, etc.

All operations are stored in oplog.



Secondary servers continuously read the oplog and apply those changes.

This process is called **Replication Sync**

What Happens if Primary Crashes?

Primary ✖ Crashed

Secondary 1

Secondary 2

Now nobody can write data.

MongoDB starts Election Process, One secondary becomes new primary.

Old Primary ✖

Secondary 1 → New Primary

Secondary 2 → Secondary

This is called **Automatic Failover**.

Replication Lag

Sometimes secondary isn't updated immediately.

Example: Primary Updated Secondary Still Syncing.

This delay is called Replication Lag.

Causes:

- ☐ Slow network
- ☐ Heavy work load
- ☐ Large writes

Read from Secondary

Normally:

Read → Primary

Write → Primary

But MongoDB allows:

Read → Secondary

Benefits:

- ❑ Less load on Primary
- ❑ Better performance

This leads to Read Preferences.

Replication vs Backup

Replication	Backup
Live copy	Stored copy
Automatic	Scheduled
High availability	Disaster recovery
Real-time sync	Point-in-time snapshot

Replication is NOT a backup.

If someone deletes data on Primary: Deleted everywhere

Primary Delete → Secondary Delete

Commands:



```
1 mongod --replSet rs0 --port 27018 --dbpath C:\Program Files\MongoDB\Server\8.2\data\db1
2 mongod --replSet rs0 --port 27019 --dbpath C:\Program Files\MongoDB\Server\8.2\data\db2
3 mongod --replSet rs0 --port 27020 --dbpath C:\Program Files\MongoDB\Server\8.2\data\db3
4
5
6 rs.initiate({
7   _id: "rs0",
8   members: [
9     { _id: 0, host: "localhost:27017" },
10    { _id: 1, host: "localhost:27018" },
11    { _id: 2, host: "localhost:27019" }
12  ]
13 })
```


Thank You